

SQL Technical Submission

Data Extraction, Advanced Querying & Performance Optimization Report

Intern Name:	Sumaiya
Role:	Data Analyst Intern
Assignment:	Week 3 Core & Advanced SQL Analytics
Environment:	MySQL (Schema: `sale_analysis`)

1. Executive Summary

This technical document comprehensive details the query builds, operational logics, and strategic business insights derived from the enterprise sales_data_sample tracking database. The deliverables fulfill requirements across filtering criteria, structural data groupings, self-joins, nested subqueries, window logic distributions, and physical layer indexing optimization routines.

2. Core SQL Queries & Functional Breakdown

Task 2.1 & 2.2: Granular Data Selection, Conditions & Logical Precedence

Filters localized market orders explicitly targeting successful shipments for the city of 'Oulu' while adding criteria to capture high-volume lines.

```
SELECT ORDERNUMBER, SALES, QUANTITYORDERED, STATUS, YEAR_ID, CITY
FROM sales_data_sample
WHERE CITY='Oulu' AND STATUS ='Shipped'
AND QUANTITYORDERED > 30
ORDER BY ORDERNUMBER DESC;
```

Strategic Business Insight: By filtering transactional volume thresholds directly on localized nodes (City: Oulu), logistics and fulfillment managers can isolate distribution patterns, ensuring bulk orders are prioritized for freight consolidation.

Task 2.3: Quantitative Fiscal Quarter Analysis

Aggregates aggregate gross income values across explicit annual sub-cycles (Quarters).

```
SELECT QTR_ID, SUM(SALES) AS Total_Sales
FROM sales_data_sample
GROUP BY QTR_ID
ORDER BY QTR_ID;
```

Strategic Business Insight: Evaluating performance trends per quarter uncovers cyclical patterns. It identifies seasonal peaks (such as end-of-year buying trends in Q4) to guide targeted demand generation efforts.

Task 2.4: Macro Corporate Performance Framework

Tracks multi-variable growth vectors (Total Revenue, Average Sales value, and total active transaction volume) on an annual scale.

```
SELECT YEAR_ID,
       SUM(SALES) AS Total_Revenue,
       AVG(SALES) AS Avg_Sales,
       COUNT(DISTINCT ORDERNUMBER) AS Total_Number
FROM sales_data_sample
GROUP BY YEAR_ID
ORDER BY YEAR_ID;
```

Strategic Business Insight: Monitoring average order values alongside overall historical trends shows whether company revenue gains are fueled by expanding consumer bases or higher price metrics per contract.

Task 2.5 & 2.6: Geopolitical Thresholding using HAVING

Extracts high-performing localized global markets contributing aggregate revenue figures above a specific \$20,000 baseline.

```
SELECT COUNTRY, SUM(SALES) AS Total_Country_sales
FROM sales_data_sample
GROUP BY COUNTRY
HAVING SUM(SALES) > 20000
ORDER BY COUNTRY;
```

Strategic Business Insight: The HAVING clause allows analysts to easily isolate top-performing territories. Markets falling below this benchmark can be quickly identified for promotional campaigns or regional pricing reviews.

3. Advanced Relational Databases & Analytical Logic

Task 3.1: Market Basket Co-occurrence Matrix (Self-Join)

Maps the primary table onto itself using an active multi-conditional internal join. It links rows matching identical order codes but holding different product IDs.

```
SELECT a.ORDERNUMBER, a.PRODUCTCODE AS Product_1, b.PRODUCTCODE AS Product_2, a.CITY
FROM sales_data_sample a
INNER JOIN sales_data_sample b
    ON a.ORDERNUMBER = b.ORDERNUMBER
    AND a.PRODUCTCODE <> b.PRODUCTCODE
LIMIT 10;
```

Strategic Business Insight: Self-joins provide a solid foundation for cross-merchandising systems. Discovering product lines that are frequently bought together allows the business to offer smart, pre-packaged item bundles.

Task 3.2: High-Margin Deviation Analytics (Subqueries)

Filters individual records dynamically by comparing transaction values against the absolute average computed at runtime.

```
SELECT ORDERNUMBER, CUSTOMERNAME, SALES
FROM sales_data_sample
WHERE SALES > (SELECT AVG(SALES) FROM sales_data_sample)
ORDER BY SALES DESC;
```

Strategic Business Insight: Isolates key transactions that exceed baseline operational averages. This gives account management quick visibility into high-value customer accounts to ensure top-tier service.

Task 3.3: Chronological Segment Distribution (Window Functions)

Deploys ROW_NUMBER() and RANK() analytical engines over annual partitions ordered by individual deal size.

```
SELECT YEAR_ID, PRODUCTLINE, SALES,
    ROW_NUMBER() OVER (PARTITION BY YEAR_ID ORDER BY SALES DESC) AS Row_Num,
    RANK() OVER (PARTITION BY YEAR_ID ORDER BY SALES DESC) AS Sales_Rank
FROM sales_data_sample;
```

Strategic Business Insight: Windowing retains granular line-item integrity while ranking product performance dynamically within each fiscal year. This avoids losing specific transaction details, unlike standard GROUP BY structures.

4. Targeted Business Problem Solving

The scripts address key operational priorities, including defining VIP customer concentration levels (Top 5 Accounts), mapping historical gross monthly revenue trends, and tracking average purchase counts to understand long-term customer buying habits.

5. Query Optimization & Schema Tuning

To improve query execution times, large unstructured text definitions were adjusted to standard VARCHAR formats. Active B-Tree tracking nodes were then added to remove costly sequential database table scans.

```
ALTER TABLE sales_data_sample MODIFY COLUMN CUSTOMERNAME VARCHAR(255);  
ALTER TABLE sales_data_sample MODIFY COLUMN CITY VARCHAR(100);  
  
CREATE INDEX idx_customer ON sales_data_sample(CUSTOMERNAME);  
CREATE INDEX idx_sales_city ON sales_data_sample(CITY);
```

Strategic Business Insight: Transitioning data fields into proper VARCHAR parameters and adding targeted indexes speeds up search lookups. This helps power analytical dashboards with sub-second response times as the data grows.